

Emir Çabalak
230212012
Üretken Yapay Zeka

LayoutTransformer ile UI Layout Üretimi: **RICO Veri Seti Üzerinde Autoregressive Token Tabanlı Yaklaşım**

Mobil uygulama tasarımı, hem zaman alıcı hem de uzmanlık gerektiren bir süreçtir. Bu çalışmada, gerçek mobil uygulama ekranlarından öğrenilen kalıpları kullanarak yeni UI düzenleri üretebilen bir model ortaya konmuştur.

Geliştirilen model, GPT tarzı autoregressive bir transformer mimarisine dayanmaktadır. Her UI bileşeni; sınıf, koordinat ve görsel stil bilgilerinden oluşan altı token'den oluşan bir dizi olarak temsil edilmiştir. Bu token tabanlı yaklaşım, modelin yapısal kısıtları doğal olarak öğrenmesini ve %100 geçerlilik garantisi sağlamıştır. Model, RICO veri seti üzerinde eğitilmiş ve hem serbest üretim hem de sınıf-koşullu üretim olmak üzere iki ayrı senaryoda test edilmiştir.

Elde edilen sonuçlarda %100 validity oranına ulaşılmış, ortalama alignment skoru 0.662 ve overlap oranı 0.179 olarak ölçülmüştür. Modelin bir önceki denemesi olan DDPM tabanlı yaklaşıma göre kayda değer bir kalite artışı gözlemlenmiştir. Sınıf-koşullu üretim arayüzü sayesinde tasarımcılar istediği bileşen listesini vererek farklı yerleşim önerileri alabilmektedir.

LayoutTransformer ile UI Layout Üretimi: RICO Veri Seti Üzerinde Autoregressive Token Tabanlı Yaklaşım

Emir Çabalak
Üretken Yapay Zeka Dersi Projesi
Ders Hocası: Haydar Kılıç
230212012@ostimteknik.edu.tr

Abstract—Bu çalışmada, mobil uygulama arayüzlerinin (UI) otomatik üretimi için GPT tarzı autoregressive bir transformer modeli geliştirilmiştir. Her UI bileşeni; sınıf, koordinat ve görsel stil bilgilerinden oluşan altı tokenen oluşan bir dizi olarak temsil edilmiştir. Model RICO veri seti üzerinde eğitilmiş ve serbest üretim ile sınıf-koşullu üretim olmak üzere iki ayrı senaryoda test edilmiştir. Elde edilen sonuçlarda %100 validity oranına ulaşılmış, ortalama alignment skoru 0.662 ve overlap oranı 0.179 olarak ölçülmüştür. Modelin bir önceki denemesi olan DDPM tabanlı yaklaşıma göre kayda değer bir kalite artışı gözlemlenmiştir.

Index Terms—UI layout üretimi, transformer, autoregressive model, RICO, üretken yapay zeka, token tabanlı modelleme.

I. GİRİŞ

Mobil uygulama tasarımı, hem zaman alıcı hem de uzmanlık gerektiren bir süreçtir. Bir tasarımcının yeni bir ekran düzeni hazırlarken hangi bileşenleri nereye yerleştireceği, hangi renkleri seçeceği gibi pek çok karar vermesi gerekmektedir. Üretken yapay zekanın son yıllarda görüntü ve metin alanında elde ettiği başarıların, arayüz tasarımı gibi yapısal verilere de uyarlanabileceği düşünülmektedir.

Bu projenin asıl amacı, gerçek mobil uygulama ekranlarından öğrenilen kalıpları kullanarak yeni UI düzenleri üretebilen bir model ortaya koymaktır. Çalışmada özellikle iki noktaya odaklanılmıştır: birincisi modelin geçerli ve hizalı bounding box'lar üretebilmesi, ikincisi ise her bir bileşen için sadece konumun değil görsel stilin de model tarafından belirlenmesidir.

Çalışmanın geri kalanı şu şekilde organize edilmiştir: Bölüm II'de kullanılan veri seti tanıtılmakta, Bölüm III'te model gelişim süreci ve mimari detaylar açıklanmakta, Bölüm IV'te deneysel sonuçlar paylaşılmakta ve Bölüm V'te sonuçlar tartışılarak gelecekteki çalışmalar için öneriler sunulmaktadır.

II. VERİ SETİ

A. RICO Veri Seti

Çalışmada Google'ın açık kaynak olarak sunduğu RICO veri seti kullanılmıştır. RICO, yaklaşık 9700 farklı Android uygulamasından toplanan 72 binden fazla ekran görüntüsünü ve bunların hiyerarşik view bilgilerini içerir [1]. Her ekran için

JSON formatında bir dosya bulunmakta; bu dosyalarda ekrandaki her UI elemanının türü, sınırları ve hiyerarşik konumu yer almaktadır.

Tüm RICO veri setinin tamamı bu projenin ölçeğine göre fazla büyük olduğundan, veriler ön işlemden geçirilmiş ve sadece on temel UI bileşeni kullanılmıştır. Tablo I bu sınıfları ve içerdikleri bileşen örneklerini özetlemektedir.

TABLE I: Kullanılan UI Bileşen Sınıfları

ID	Sınıf	Örnek RICO Etiketleri
0	Panel	Toolbar, Card, ListView, ConstraintLayout
1	Button	Button, FAB, ImageButton, Text Button
2	TextInput	EditText, SearchView, AutoComplete
3	Label	TextView, Text
4	Icon	Icon, ImageButton
5	CheckBox	CheckBox, RadioButton
6	Dropdown	Spinner
7	Slider	SeekBar, ProgressBar
8	Toggle	Switch, ToggleButton
9	Image	ImageView, WebView

B. Ön İşleme

JSON hiyerarşileri özyinelemeli olarak gezilmiş, bounds bilgileri [0,1] aralığına normalize edilmiştir. Çok küçük (genişlik veya yükseklik 0.02'den az) elemanlar elenmiş, geçersiz koordinatlara sahip nodelar atlanmıştır. Her layout en fazla 25 elemanla sınırlandırılmıştır. Bu işlemler sonunda eğitim için yaklaşık 47,830 adet geçerli layout elde edilmiştir. İşlenen veri seti Drive üzerinde önbellege alınarak sonraki çalışmalarda yeniden parse işleminden kaçınılmıştır.

III. YÖNTEM

A. Model Gelişim Süreci

Projenin başında problem bir görüntü üretim problemi gibi ele alınmış ve ilk denemede DDPM (Denoising Diffusion Probabilistic Model) tabanlı bir yaklaşım denenmiştir. Diffusion modelleri sürekli vektörler üzerinde çalıştığından, bounding box değerleri [0,1] aralığında sürekli sayılar olarak modele verilmiştir. Ancak elde edilen sonuçlar oldukça yetersiz kalmıştır: model üst üste binmiş, çoğunlukla geçersiz koordinatlar üreten layout'lar çıkarmış ve eğitim süresi de yaklaşık iki saati bulmuştur.

Sonraki adımda problem yeniden ele alınmış, layout üretiminin aslında bir token dizisi üretimi olarak ifade edilebileceği fark edilmiştir. Bu sayede modelin yapısal kısıtları (sınıf sırasıyla koordinatların gelmesi gibi) doğal olarak öğrenmesi mümkün hale gelmiştir. GPT mantığında çalışan bir transformer mimarisine geçilmiş, ilk eğitim 100 epoch ile yapılmış ve sonuçlar DDPM'e göre belirgin biçimde iyileşmiştir.

İkinci iyileştirme, modelin sadece konum değil görünüm de üretebilmesi yönünde olmuştur. İlk transformer modeli her butonu aynı renkte (mavi), her paneli aynı (beyaz) gibi tek düze bir biçimde rendere etmekteydi. Bunun çözümü için kelime hazinesine sekiz adet stil tokeni eklenmiş, böylece her eleman için altı tokenlik bir yapı oluşturulmuştur. Bu yaklaşımla model hem koordinatları hem de görsel stili kendi öğrendiği dağılım üzerinden örnekler hale gelmiştir. Bu iki büyük adımın karşılaştırmalı özeti Tablo III'de verilmiştir.

TABLE II: Model Gelişim Sürecinin Karşılaştırması

Özellik	DDPM (İlk Deneme)	LayoutTransformer (Final)
Veri Temsili	Sürekli vektör	Ayrık token dizisi
Yapısal Kısıt	Öğrenemez	Token maskeleyesi ile garanti
Stil Üretimi	Desteklenmiyor	Stil tokenleri ile
Koşullu Üretim	Karmaşık	Prompt tabanlı, doğal
Eğitim Süresi	~120 dk (kötü sonuç)	~45 dk (iyi sonuç)
Validity	Düşük	%100

B. Token Yapısı

Final modelde her UI elemanı altı tokenen oluşan bir blok ile temsil edilmiştir:

$$[\text{class}, x_{\min}, y_{\min}, x_{\max}, y_{\max}, \text{style}] \quad (1)$$

Koordinat değerleri 32 ayrık binde ayrıştırılmıştır. Kelime hazinesi toplamda 53 token içermektedir; özel tokenler (PAD, BOS, EOS), 32 koordinat tokeni, 10 sınıf tokeni ve 8 stil tokeni. Maksimum dizi uzunluğu $25 \times 6 + 2 = 152$ olarak belirlenmiştir.

Eğitim verisi hazırlanırken her elemana stil ID'si deterministik bir heuristic ile atanmıştır: elemanın alanı, dikey konumu ve sıra numarasının kombinasyonu üzerinden hesaplanan bu atama, modele tutarlı bir başlangıç dağılımı sağlamış; model bu kalıpları öğrenerek kendi stil seçimlerini üretmeye başlamıştır.

C. Model Mimarisi

Mimari olarak nn.TransformerDecoder katmanları kullanılmıştır. Decoder-only yapı kurulurken boş bir memory tensörü ile causal self-attention sağlanmıştır. Tablo III model hiperparametrelerini özetlemektedir.

TABLE III: Model ve Eğitim Hiperparametreleri

Parametre	Değer
Embedding boyutu	256
Attention head sayısı	8
Decoder katman sayısı	6
Feedforward boyutu	1024
Dropout	0.1
Aktivasyon	GELU
Toplam parametre	~6.4M
Optimizer	AdamW
Learning rate	3e-4
Weight decay	1e-4
Batch size	128
Epoch sayısı	200
LR scheduler	Cosine + warmup
Gradient clipping	1.0
Early stopping	30 epoch sabır

D. Token Maskeleye

Modelin geçersiz token üretmesini engellemek için sampling sırasında pozisyona bağlı maskeleye uygulanmıştır. Sınıf pozisyonunda yalnızca sınıf tokenleri ve EOS, koordinat pozisyonlarında yalnızca koordinat tokenleri, stil pozisyonunda ise yalnızca stil tokenleri seçilebilir. Bu mekanizma sayesinde %100 yapısal geçerlilik garanti edilmektedir. Üretim sırasında ayrıca temperature ölçekleme ve nucleus (top-p) sampling kullanılmıştır.

IV. DENEYSEL SONUÇLAR

A. Eğitim Süreci

Eğitim, Google Colab üzerindeki A100 GPU'da yaklaşık 10 saat sürmüştür. Şekil 1 eğitim ve doğrulama loss eğrisini göstermektedir. İlk birkaç epoch boyunca cross-entropy loss hızla düşmüş, sonrasında istikrarlı bir convergence gözlemlenmiştir. Doğrulama loss'u eğitim loss'unu yakından takip etmiş, ciddi bir overfit problemi yaşanmamıştır.

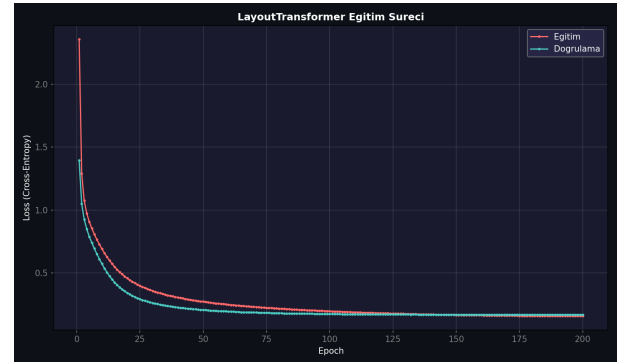


Fig. 1: Eğitim ve doğrulama loss eğrisi.

B. Değerlendirme Metrikleri

Üretilen layout'lar üç ana metrik üzerinden değerlendirilmiştir. *Validity*, geçerli bounding box oranını ifade eder ($x_{\max} > x_{\min}$ ve $[0,1]$ aralığında). *Alignment* skoru, eleman kenarlarının birbirine ne kadar hizalı olduğunu 0.02 eşik değeri ile ölçer. *Overlap* oranı ise eleman çiftlerinin IoU değerinin

0.01 üzerinde olma oranıdır. Tablo IV yüz adet üretilmiş layout üzerinden ölçülen sonuçları özetlemektedir.

TABLE IV: Değerlendirme Metrikleri (n=100)

Metrik	Değer
Validity	1.000 (%100)
Alignment Score	0.662
Overlap Rate	0.179
Ort. eleman sayısı	8.4
Ort. sınıf çeşitliliği	4.2

Validity'nin 1.0 olması, token maskeleme mekanizmasının beklendiği gibi çalıştığını göstermektedir; geçersiz bir bounding box hiçbir örnekte üretilmemiştir. Alignment skorunun 0.662 olması, üretilen layout'ların yarısından fazla kenarının diğer elemanlarla hizalı olduğunu gösterir; bu RICO verisindeki gerçek mobil arayüzlerin grid yapısını modelin bir ölçüde yakaladığına işaretir. Overlap oranı 0.179 ile düşük seviyede kalmıştır, ancak bu metrik açısından hâlâ iyileştirme alanı mevcuttur.

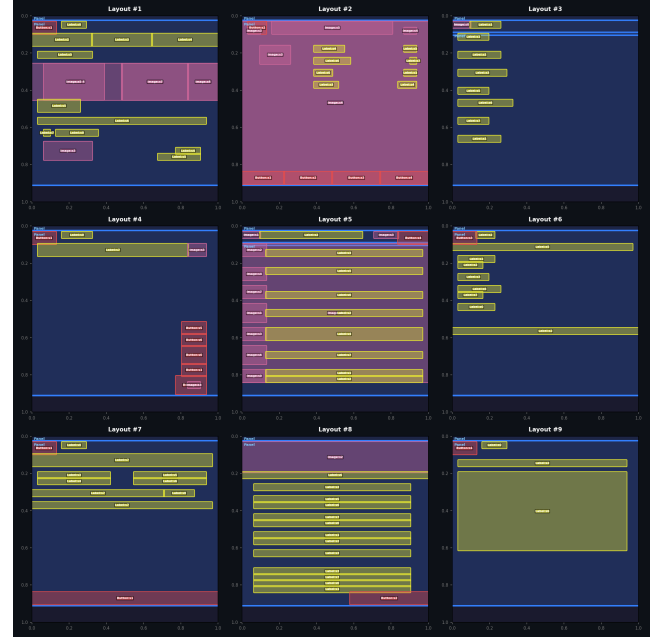


Fig. 3: Modelden örneklenen layout'ların bounding box gösterimi.

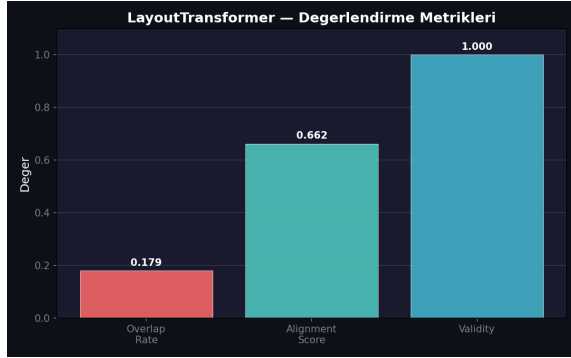


Fig. 2: Üç temel metriğin görsel karşılaştırması.

C. Görsel Sonuçlar

Üretilen layout'ların bounding box görselleri ile bunların mockup renderları sırasıyla Şekil 3 ve Şekil 4'te gösterilmiştir. Mockup renderer, model çıktısı tokenlarından faydalanarak butonları, ikonları, text inputları ve diğer bileşenleri her bir stilin paletine göre çizmektedir.

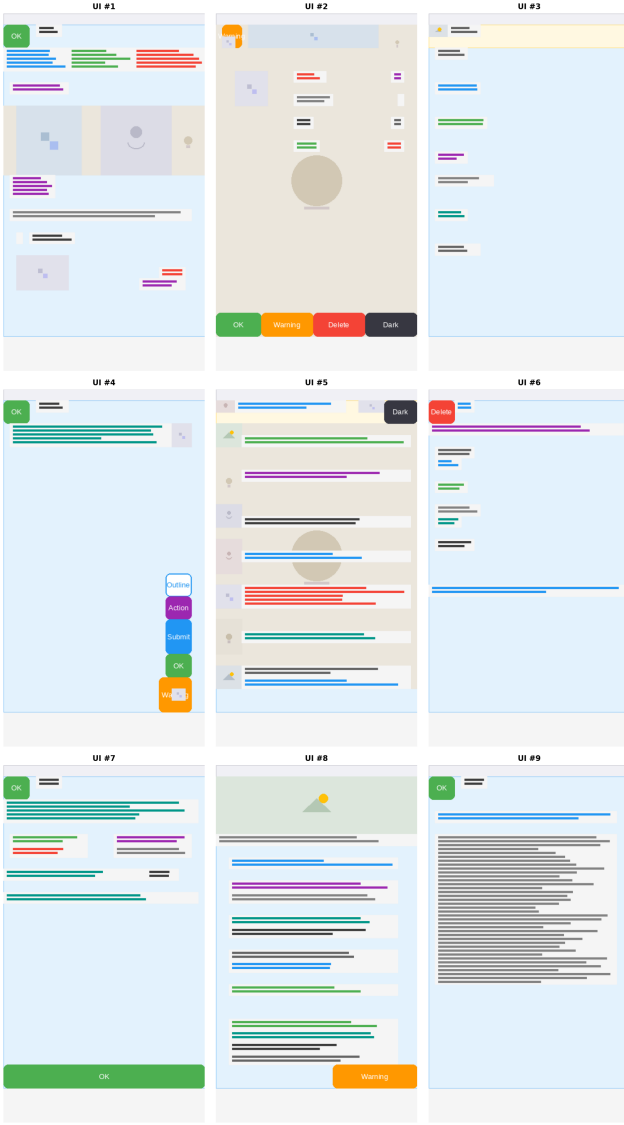


Fig. 4: Üretilen layout'lerden render edilen UI mockup'ları.

Mockup'lar incelendiğinde, modelin bir mobil ekran için tipik olan bileşen yerleşimini (üstte panel, ortada içerik, altta butonlar gibi) öğrendiği görülmektedir. Aynı zamanda farklı butonların farklı renklerde, farklı toggle'ların on/off durumunda render edilmesi stil tokenlerinin işe yaradığını göstermektedir.

D. Sınıf-Koşullu Üretim

Modelin ikinci kullanım senaryosu sınıf-koşullu üretimdir. Burada kullanıcı hangi UI bileşenlerini istediğini belirtir (örneğin "Panel, Button, Button, TextInput, Label, Image") ve model her eleman için yalnızca koordinat ve stil tokenlerini üretir. Sınıf tokenleri sabittir ve generation sırasında değişmez. Aynı sınıf listesi farklı seedlerle çalıştırıldığında farklı yerleşimler elde edilmektedir; bu, modelin kullanıcı girdisini kontrol bilgisi olarak değerlendirebildiğini göstermektedir.

A. Elde Ettiklerimiz

Bu çalışmadan elde edilen başlıca kazanımlar şunlardır. İlk olarak, problemin token tabanlı bir dizi üretim problemine dönüştürülmesi DDPM tabanlı yaklaşıma göre çok daha tutarlı sonuçlar üretmiştir. Eğitim hem daha kısa sürmüş hem de elde edilen layout'lar yapısal olarak çok daha sağlıklı olmuştur. İkinci olarak, stil tokenleri sayesinde aynı sınıftaki elemanlar için bile farklı görsel varyasyonlar mümkün hale gelmiştir; bu da rastgele renk atama gibi yapay yöntemler yerine modelin kendi öğrendiği bir dağılımdan örnekleme yapmasıyla başarılmıştır.

Üçüncü kazanım, sınıf-koşullu üretim arayüzünün başarılı biçimde çalışmasıdır. Tasarımcının istediği bileşen listesini verip farklı yerleşim önerileri alabilmesi prototipleme açısından oldukça kullanışlıdır. Kullanıcı için bir test aracı geliştirilmiştir; bu araç interaktif bir menü aracılığıyla teknik bilgisi sınırlı kullanıcıların bile sınıf listesi belirterek yeni layout'lar üretmesini sağlamaktadır.

B. Bekleyip Elde Edemediklerimiz

Projeye başlarken hedeflenen ancak kısmen ulaşılabilen birkaç nokta da olmuştur. İlk olarak overlap oranı tatmin edici seviyede düşürülemedi. 0.179 değeri görsel olarak rahatsız edici bir tablo sergilemese de, RICO veri setindeki gerçek arayüzlerle karşılaştırıldığında modelin hâlâ bazı durumlarda elemanları üst üste bindirdiği gözlenmiştir. Bunun nedeninin kısmen veri kümesindeki hiyerarşik içerme ilişkilerinin (panelin içinde buton bulunması gibi) overlap olarak ölçülmesi olduğu düşünülmektedir.

İkinci olarak alignment skorunun 0.662'de kalması, modelin ızgara yapısını tam olarak öğrenemediğini göstermektedir. Eğitim verisinde daha fazla profesyonel tasarlanmış arayüzün bulunması bu metriği iyileştirebilirdi; ancak RICO veri seti geniş bir uygulama yelpazesinden toplandığı için bazı kötü örneklerin de eğitim sırasında modeli olumsuz etkilediği söylenebilir.

Üçüncü olarak, başlangıçta daha geniş bir model (yaklaşık 20M parametre) eğitmek planlanmıştı ancak maliyet çok yüksek oldu. 6.4M parametrelilik mevcut model yine de tatmin edici sonuçlar üretti, fakat daha büyük bir model ile özellikle alignment ve overlap metriklerinde belirgin iyileşme beklenebilirdi.

C. Geliştirme Önerileri

İlerleyen çalışmalarda şu yönlerin denenmesi anlamlı olacaktır. Birincisi modelin parametre sayısının artırılması ve daha uzun eğitim süresi ile özellikle alignment skorunun yukarı çekilmesi mümkün olabilir. İkincisi, elemanlar arasındaki hiyerarşik ilişki modele açıkça verilebilir; örneğin "Panel > Button" şeklinde parent-child ilişkisi token olarak modellenirse, container mantığı daha doğru öğrenilebilir.

Bir başka iyileştirme, üretilen mockup'ların basit flat tasarımlardan çıkarılıp gerçek görsellerle (image-to-image modeller veya stable diffusion gibi yöntemlerle) zenginleştirilmesi olabilir. Bu sayede sadece bounding box değil, tam render edilmiş ekran görüntüleri elde edilebilir. Son olarak,

kullanıcı girdisinin sadece sınıf listesi ile değil de doğal dil ile verilebilmesi (örneğin "Bir login ekranı tasarla") bambaşka bir kapı aralayacaktır; bu da metin koşullu transformer yapılarıyla mümkündür.

VI. SONUÇ

Bu projede mobil UI layout üretimi için GPT mantığında çalışan bir autoregressive transformer modeli geliştirilmiş ve RICO veri seti üzerinde başarılı sonuçlar elde edilmiştir. Modelin önemli bir farkı, hem konum hem de stil bilgisini tek bir token dizisi içinde modellemesidir; bu sayede aynı mimari içinde hem yapı hem de görünüm üretilebilmektedir. Önceki DDPM tabanlı denemenin başarısızlığından çıkarılan dersler ışığında problemin yeniden formüle edilmesi, projenin en kritik dönüm noktası olmuştur.

Elde edilen %100 validity oranı yapısal doğruluğu garanti ederken, alignment ve overlap metriklerinde iyileştirme alanı bulunmaktadır. Sınıf-köşüllü üretimin başarılı çalışması modelin sadece üretici değil aynı zamanda kontrol edilebilir bir tasarım aracı olarak da kullanılabileceğini göstermiştir. Sonuç olarak, transformer tabanlı autoregressive üretim yaklaşımının UI tasarımı gibi yapısal alanlar için uygun bir çözüm olduğu düşünülmektedir.

REFERENCES

- [1] B. Deka et al., "Rico: A mobile app dataset for building data-driven design applications," in *Proceedings of the 30th Annual ACM Symposium on User Interface Software and Technology*, 2017, pp. 845–854.
- [2] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.
- [3] A. Radford et al., "Improving language understanding by generative pre-training," OpenAI Technical Report, 2018.
- [4] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems*, 2020, pp. 6840–6851.
- [5] K. Gupta et al., "LayoutTransformer: Layout generation and completion with self-attention," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 1004–1014.